

Rapport de stage d'activités professionnelles

Bachelier informatique de gestion

Gaucher Adrien

2025-2026



1. Remerciements

Je tiens à remercier l'ensemble des personnes qui m'ont accompagné durant mon stage au sein du Chimps Lab.

Je remercie tout particulièrement mon maître de stage pour son encadrement, sa disponibilité et ses conseils tout au long de ces trois mois. Son suivi régulier m'a permis de progresser aussi bien sur le plan technique que sur le plan méthodologique, et de mieux comprendre les exigences liées à un projet professionnel réel.

Enfin, je remercie mon établissement et l'ensemble des enseignants de ma formation, qui m'ont fourni les bases théoriques et pratiques nécessaires pour aborder ce stage dans de bonnes conditions. Ce stage représente une étape importante dans mon parcours, en me permettant de mettre en pratique les compétences acquises durant mon bachelier en informatique.

2. Présentation de l'entreprise

2.1 Présentation générale

Chimps Lab est une structure technologique et créative qui combine développement logiciel, production multimédia et expérimentation technique. L'entreprise est centrée autour d'un environnement de travail polyvalent permettant de concevoir et tester des projets dans des domaines variés, allant du développement informatique à la création de contenus interactifs.

L'activité principale de Chimps Lab repose sur le développement de solutions logicielles sur mesure ainsi que sur des projets liés aux technologies immersives et interactives. L'entreprise intervient notamment dans des domaines tels que le développement de logiciels, les systèmes de gestion, ainsi que des projets liés aux moteurs de jeux vidéo / rendu 3D et aux environnements temps réel.

Le fondateur de la structure possède une expérience initiale en graphics programming et en développement sur moteurs de jeu (notamment Unreal Engine). En parallèle, il développe également des solutions logicielles orientées gestion, notamment pour des structures éducatives.

L'environnement de Chimps Lab est également particulier dans sa conception : il s'agit d'un espace hybride regroupant à la fois un espace de développement informatique, un studio équipé (fond vert, régie vidéo), ainsi que des espaces de travail collaboratif et des zones de réunion. Cette configuration permet de soutenir des projets techniques et créatifs dans un même lieu.

2.2 Organisation interne

Chimps Lab est une structure de petite taille, centrée principalement autour de son fondateur, qui assure à la fois les rôles de développeur, chef de projet et responsable technique. Selon les projets, des collaborateurs externes ou stagiaires peuvent être intégrés afin de renforcer l'équipe sur des tâches spécifiques.

Dans le cadre de mon stage, j'ai été encadré directement par le fondateur de l'entreprise, qui a assuré le rôle de maître de stage. Son suivi s'est fait de manière continue, avec des échanges réguliers permettant de valider l'avancement des tâches, d'orienter les choix techniques et de s'assurer de la cohérence globale du projet.

La méthode de travail adoptée était basée sur une organisation flexible et orientée projet. Le suivi des tâches se faisait de manière itérative, avec des points réguliers permettant d'ajuster les priorités en fonction de l'évolution du développement. La communication était directe et informelle, favorisant la réactivité et l'adaptation rapide aux besoins du projet.

2.3 Infrastructure

L'infrastructure du Chimps Lab repose sur un ensemble de technologies permettant le développement, l'hébergement et l'exploitation de différents services internes et applicatifs.

L'entreprise s'appuie sur une architecture distribuée composée de plusieurs services distincts. Cette séparation permet d'isoler les différents usages tels que le développement, la gestion de projet, le déploiement applicatif ou encore les traitements plus spécifiques comme le rendu ou le streaming.

Plusieurs serveurs sont utilisés au sein de l'infrastructure interne, chacun ayant un rôle dédié. Certains sont utilisés pour l'hébergement des outils de développement et de collaboration, comme Git et Planka, tandis que d'autres sont dédiés à l'exécution de services applicatifs ou de traitements plus lourds, notamment liés au traitement vidéo.

Cette organisation permet une meilleure répartition des charges et une plus grande flexibilité dans l'utilisation des ressources.

L'environnement réseau de l'entreprise est structuré de manière professionnelle, avec une infrastructure filaire basée principalement sur des connexions Ethernet (RJ45), garantissant une bonne stabilité et de bonnes performances pour les services hébergés localement.

Sur le plan du développement, le code source est géré via Git, tandis que la gestion de projet est assurée à l'aide de Planka. L'ensemble contribue à un environnement de travail structuré et cohérent avec les besoins des projets techniques de l'entreprise.

3. Contexte et objectifs du projet

3.1 Le projet

Le projet auquel j'ai participé consiste en la conception d'un système de contrôle d'accès intelligent destiné à être intégré à une plateforme de réservation d'espaces. L'objectif est de permettre aux utilisateurs de réserver un ou plusieurs espaces via une interface web, puis d'y accéder de manière autonome grâce à un système de serrures connectées.

Contrairement à un système de clés physiques traditionnel, la solution développée repose sur une gestion entièrement numérique des accès. Lorsqu'une réservation est validée, un QR code unique est généré et associé à celle-ci. Ce QR code constitue la clé d'accès de l'utilisateur et lui permet d'ouvrir uniquement les portes correspondant à sa réservation, pendant la période de validité définie.

Le système est composé de plusieurs éléments complémentaires. Une application web permet l'administration des espaces, des utilisateurs et des réservations. Un serveur central assure la gestion des droits d'accès, la communication avec les serrures connectées ainsi que l'enregistrement des événements. Enfin, chaque serrure est pilotée par un microcontrôleur chargé d'exécuter les commandes reçues et d'actionner le mécanisme de verrouillage.

Cette architecture centralisée permet d'administrer l'ensemble des accès depuis une interface unique. Elle offre également une meilleure traçabilité des opérations grâce à l'enregistrement des événements, ce qui facilite le suivi des accès et le diagnostic d'éventuels dysfonctionnements.

Le projet est conçu de manière modulaire afin de pouvoir être adapté à différents contextes d'utilisation, tels que la location de salles de réunion, de studios, d'espaces de coworking ou de tout autre local nécessitant une gestion automatisée des accès.

3.2 Objectifs

Au-delà de la réalisation d'un produit fonctionnel, le projet avait pour objectif de valider la faisabilité technique d'une solution complète de contrôle d'accès connecté.

Le premier objectif était de démontrer qu'il est possible d'intégrer un système de serrures intelligentes directement dans le processus de réservation d'espaces, en remplaçant les clés physiques par des autorisations numériques.

Le prototype devait également permettre de définir une architecture logicielle évolutive, capable d'accueillir de nouvelles fonctionnalités telles que la gestion de plusieurs sites, l'ajout de nouveaux types de serrures ou encore l'intégration avec d'autres services de réservation.

Un autre objectif consistait à expérimenter différentes technologies afin d'identifier les solutions les plus adaptées au projet. Le développement s'appuie ainsi sur un microcontrôleur ESP32 pour la partie embarquée, un backend développé en Node.js, une base de données MongoDB pour le stockage des informations et une interface web réalisée avec Vue.js.

Enfin, ce prototype devait servir de base à de futurs développements en validant la communication entre les différents composants du système, la fiabilité des échanges et le bon fonctionnement de l'ensemble de la chaîne, depuis la réservation d'un espace jusqu'à l'ouverture effective de la serrure.

4. Organisation du stage et méthodologie de travail

Tout au long du stage, l'organisation du travail s'est inspirée des méthodes agiles, tout en restant adaptée à la taille de l'entreprise et à l'équipe de développement. L'objectif était de favoriser une communication régulière, un suivi continu de l'avancement et une adaptation rapide aux besoins du projet.

chaque journée débutait par un court échange avec mon maître de stage. Ce point quotidien permettait de faire le bilan du travail réalisé la veille, de définir les objectifs de la journée et de discuter des éventuelles difficultés rencontrées. Il constituait également un moment d'échange sur les choix techniques à adopter et permettait de réorienter certaines tâches si nécessaire en fonction de l'avancement du projet.

En complément de ces points quotidiens, une réunion hebdomadaire était organisée avec le maître de stage afin de faire le bilan des travaux réalisés, d'évaluer l'avancement du projet et de définir les priorités pour les jours suivants. Ces réunions constituaient également un moment privilégié pour discuter des choix d'architecture, des solutions techniques envisagées et des améliorations possibles.

La gestion du code source était assurée à l'aide de Git, hébergé sur une instance Forgejo installée sur l'infrastructure interne de l'entreprise. Cette plateforme permettait de centraliser l'ensemble des dépôts du projet, de suivre l'historique des modifications et de faciliter la collaboration. Les nouvelles fonctionnalités étaient développées sur des branches dédiées avant d'être intégrées au projet principal après relecture.

Le suivi des tâches était réalisé à l'aide de Planka, un outil de gestion de projet inspiré de la méthode Kanban. Chaque fonctionnalité, correction de bug ou amélioration faisait l'objet d'une carte décrivant le travail à réaliser. Les tâches étaient organisées selon leur état d'avancement (*À faire*, *En cours*, *Terminé*), offrant une vision claire de la progression du projet.

Afin d'assurer une meilleure traçabilité, chaque tâche était également associée à une branche Git identifiée par un numéro de version (par exemple *V001*, *V002*, etc.). Les cartes étaient en outre catégorisées selon leur scope dans le projet (firmware, backend..). Cette organisation facilitait le suivi des développements, la gestion des priorités ainsi que le lien entre les évolutions du code et les fonctionnalités implémentées.

Le tableau était mis à jour aussi bien par mon maître de stage que par moi-même, ce qui permettait de suivre l'avancement du projet en temps réel et d'assurer une bonne coordination tout au long du développement.

Cette méthodologie de travail m'a permis d'évoluer dans un environnement proche des pratiques professionnelles actuelles. L'utilisation conjointe d'un système de gestion de versions, d'un outil de suivi des tâches et de réunions régulières a favorisé une organisation efficace, une meilleure communication et un développement progressif des différentes fonctionnalités du projet.

5. Analyse du projet

5.1 État de l'art

Positionnement par rapport aux solutions existantes

Le contrôle d'accès numérique repose aujourd'hui sur plusieurs approches courantes. Les badges RFID/NFC, très utilisés dans les environnements professionnels, nécessitent la distribution d'un support physique, ce qui est moins adapté à un accès ponctuel lié à une réservation.

Les serrures connectées grand public, comme Nuki ou August, proposent des fonctionnalités proches de celles recherchées dans ce projet. Elles sont cependant conçues comme des produits autonomes destinés à couvrir un ensemble de cas d'utilisation génériques. Notre approche vise au contraire à intégrer directement le contrôle d'accès dans une plateforme de réservation, avec un fonctionnement et un modèle de données définis spécifiquement pour ce contexte.

Notre objectif était de développer une solution directement adaptée à notre plateforme de réservation, tout en conservant la maîtrise de son fonctionnement. Le choix du QR code permet à l'utilisateur de présenter son pass depuis son smartphone, sans matériel supplémentaire. Le système permet également à un administrateur de déclencher l'ouverture d'une serrure à distance depuis le tableau de bord.

Un autre objectif important était de ne pas dépendre d'un écosystème propriétaire pour le contrôle d'accès. Le développement de notre propre infrastructure permet de maîtriser le fonctionnement du système ainsi que le stockage des données liées aux utilisateurs, aux accès et aux événements. Il permet également d'adapter le système aux besoins spécifiques de la plateforme de réservation.

Le prototype se positionne ainsi comme une solution de contrôle d'accès développée spécifiquement pour un système de réservation d'espaces professionnels, avec une maîtrise du matériel, du logiciel et des données.

5.2 Analyse des besoins

Le projet de serrures connectées s'inscrit dans un contexte de location d'espaces à accès temporaire, où les droits d'entrée sont strictement limités dans le temps et associés à une réservation. L'objectif est de permettre à un utilisateur d'accéder de manière autonome à un espace physique réservé, sans intervention humaine, grâce à un système de contrôle d'accès entièrement numérique.

Acteurs du système

- **Administrateur** — gère les zones, les serrures et les utilisateurs depuis l'interface web ; consulte l'historique des événements et diagnostique les anomalies.
- **Utilisateur final** — dispose d'un pass d'accès (QR code) associé à une réservation ; n'interagit avec le système qu'au moment de présenter son QR code devant une serrure.
- **Serrure connectée** — dispositif embarqué autonome qui valide les demandes d'accès auprès du serveur central et exécute l'ouverture physique.

Besoins fonctionnels

Besoin	Description
Gestion des zones	Créer, modifier et organiser des zones représentant des espaces physiques distincts, chacune regroupant une ou plusieurs serrures
Génération de pass	Générer un QR code unique (UUID) associé à une ou plusieurs zones, avec période de validité définie
Activation / désactivation d'un pass	Permettre la désactivation ou réactivation manuelle d'un pass avant l'expiration de sa période de validité
Validation d'accès	Vérifier, à chaque présentation d'un QR code, l'existence du pass, sa validité temporelle et son association avec la zone concernée
Initialisation d'une serrure	Associer un dispositif physique à une zone lors de sa mise en service, via un processus de configuration dédié
Journalisation	Enregistrer chaque tentative d'accès, ouverture réussie ou refus dans un journal d'événements consultable par l'administrateur
Suivi en temps réel	Refléter instantanément l'état du système et des événements sur le tableau de bord d'administration
Ouverture a distance	Permettre à un administrateur de déclencher l'ouverture d'une serrure à distance afin de conserver un contrôle étendu sur les accès.

Besoins non fonctionnels

Critère	Exigence
Sécurité	Authentifier les communications entre les serrures et le serveur à l'aide d'un token afin de limiter les accès non autorisés.
Performance	Garantir un temps de réponse suffisamment faible pour permettre une ouverture quasi immédiate après validation de l'accès.
Fiabilité	Assurer un fonctionnement stable de la serrure et une gestion correcte des erreurs, notamment en cas de perte ou d'instabilité du réseau.
Disponibilité	Permettre à la serrure de reprendre automatiquement son fonctionnement après une coupure réseau ou un redémarrage.
Maintenabilité	Concevoir le système de manière suffisamment modulaire pour faciliter son évolution et le remplacement de ses différents composants.
Utilisabilité	Permettre la configuration et l'utilisation de la serrure sans nécessiter de connaissances techniques particulières de la part de l'utilisateur.

5.3 Choix d'architecture

Après avoir identifié les besoins fonctionnels et non fonctionnels, plusieurs choix d'architecture ont été réalisés afin de concevoir un système répondant aux contraintes du projet. Ces choix concernent principalement la gestion des accès, les mécanismes d'authentification, le support d'identification des utilisateurs ainsi que le mode de communication entre les différents composants.

5.3.1 Architecture centralisée

Le prototype repose sur une architecture centralisée dans laquelle l'ensemble des droits d'accès est stocké sur un serveur unique. Les serrures ne conservent localement que leur configuration (ip du serveur, token d'authentification et paramètres réseau) et sollicitent le serveur pour chaque demande d'ouverture.

Cette approche présente plusieurs avantages. Elle permet tout d'abord la révocation immédiate d'un droit d'accès sans nécessiter de synchronisation avec les serrures déjà déployées. Elle simplifie également l'administration du système en regroupant l'ensemble des informations au sein d'une base de données unique. Enfin, elle facilite la journalisation des événements en conservant un historique complet de toutes les tentatives d'accès.

En contrepartie, cette architecture introduit une dépendance à la disponibilité du réseau et du serveur central. Une interruption de la communication empêche la validation des nouvelles demandes d'accès. Cette limitation a été jugée acceptable dans le cadre du prototype.

5.3.2 Authentification des serrures

Chaque serrure possède un identifiant unique ainsi qu'un token d'authentification généré lors de son enregistrement dans le système. Ce token est transmis à la serrure lors de sa configuration initiale puis utilisé pour toutes les communications avec le serveur.

L'utilisation d'un token permet au serveur de vérifier l'identité de la serrure à l'origine de chaque requête. La validation d'un accès ne repose donc pas uniquement sur le QR code présenté par l'utilisateur : le serveur vérifie également que la demande provient bien d'un dispositif enregistré et autorisé. Cette double vérification renforce la sécurité du système et limite les risques d'usurpation.

5.3.3 Utilisation des QR codes

Plusieurs technologies d'identification auraient pu être envisagées, notamment les badges RFID/NFC, le Bluetooth ou encore les claviers à code.

Le QR code a été retenu car il ne nécessite aucun matériel spécifique côté utilisateur. Un simple smartphone suffit pour présenter un pass d'accès. Cette solution simplifie également la distribution des accès temporaires, puisqu'un QR code peut être généré et transmis numériquement sans intervention physique.

Le même principe a également été appliqué à la configuration des serrures. Plutôt que d'utiliser une interface de configuration complexe ou une connexion filaire, les paramètres réseau et les informations d'authentification sont transmis au moyen de QR codes affichés depuis l'interface d'administration. Le QR code devient ainsi une véritable interface homme-machine (IHM), simplifiant considérablement l'installation des dispositifs.

5.3.4 Communication client-serveur

Chaque demande d'accès suit un modèle de communication de type requête/réponse. Lorsqu'un utilisateur présente un QR code, la serrure transmet les informations nécessaires au serveur, qui réalise l'ensemble des vérifications avant de renvoyer sa décision.

Ce fonctionnement garantit que toutes les décisions d'autorisation sont prises à partir des données les plus récentes disponibles dans la base de données. Les modifications apportées aux utilisateurs, aux droits d'accès ou aux serrures sont ainsi prises en compte immédiatement, sans mécanisme de synchronisation supplémentaire entre les différents équipements.

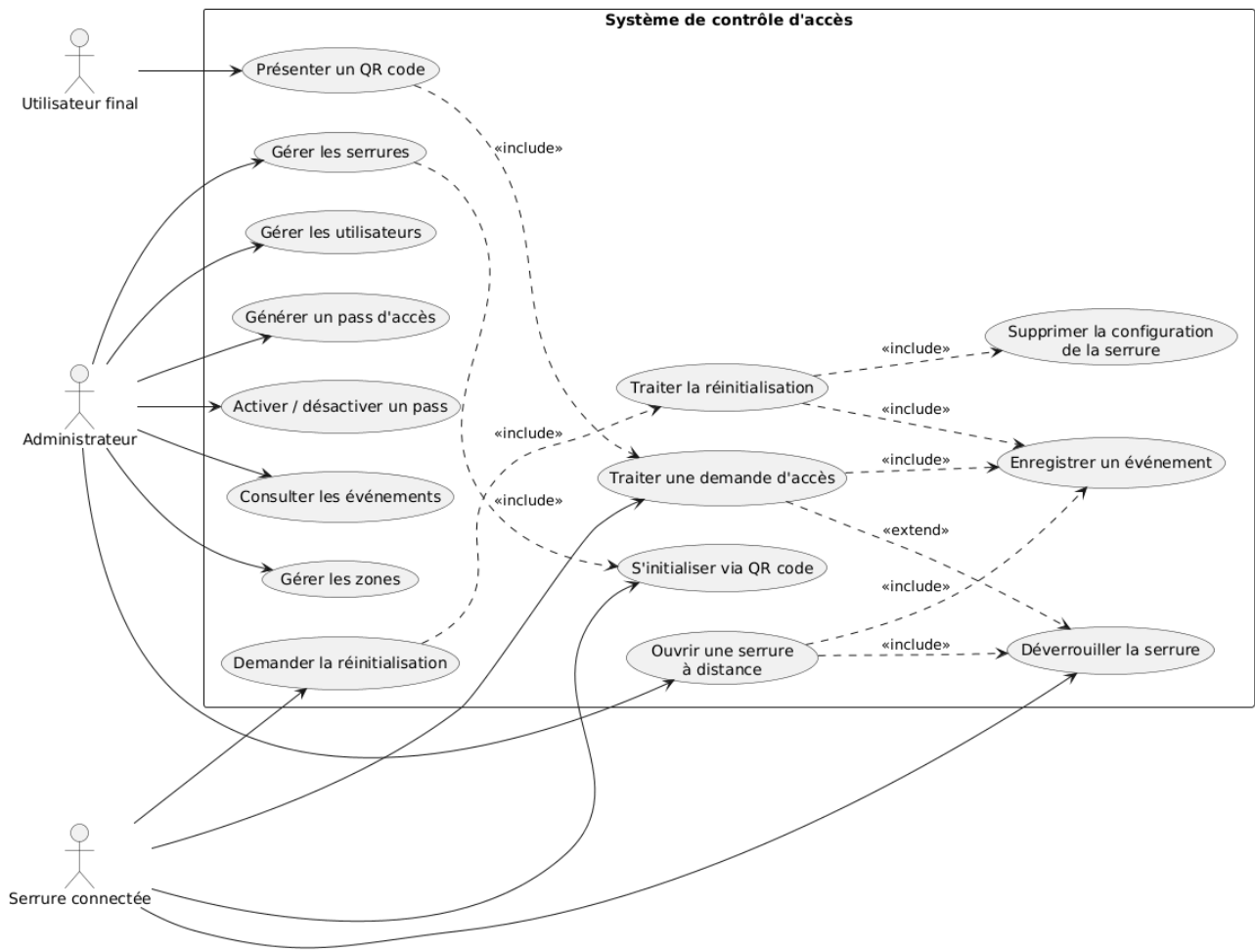
5.4 Cas d'utilisation

Afin de formaliser les interactions entre les différents acteurs et le système, un diagramme de cas d'utilisation a été réalisé (Figure X). Celui-ci met en évidence les principales fonctionnalités offertes par la plateforme, aussi bien pour les administrateurs que pour les utilisateurs finaux et les serrures connectées.

L'administrateur assure la gestion des zones, des utilisateurs, des serrures et des passes d'accès. Il peut également consulter les événements enregistrés et déclencher l'ouverture d'une serrure à distance lorsque cela est nécessaire.

L'utilisateur final interagit uniquement avec la serrure en présentant son QR code afin de demander l'accès à un espace réservé.

Enfin, la serrure connectée constitue un acteur à part entière du système. Elle est responsable de son initialisation, de la transmission des demandes d'accès au serveur, de l'exécution des commandes d'ouverture ainsi que de la notification de sa réinitialisation.



6. Développement

6.1 Développement hardware

Le développement de la partie matérielle a consisté à concevoir un prototype de serrure connectée capable d'assurer l'identification d'un utilisateur par QR code, de communiquer avec le serveur central et de commander l'ouverture d'une serrure électromagnétique.

Dans un premier temps, l'ensemble du prototype a été réalisé sur une plaque d'essai (*breadboard*). Cette approche a permis de modifier facilement le câblage au fur et à mesure de l'avancement du développement, de remplacer rapidement certains composants en cas de besoin et de faciliter les phases de débogage.

Le cœur du système repose sur un microcontrôleur ESP32-CAM, retenu pour sa connectivité Wi-Fi intégrée ainsi que pour la présence d'une caméra permettant la lecture des QR codes sans nécessiter de matériel supplémentaire. Le décodage des QR codes est assuré à l'aide de la bibliothèque ESP32QRCodeReader, qui exploite directement les images acquises par la caméra afin d'extraire les informations d'identification.

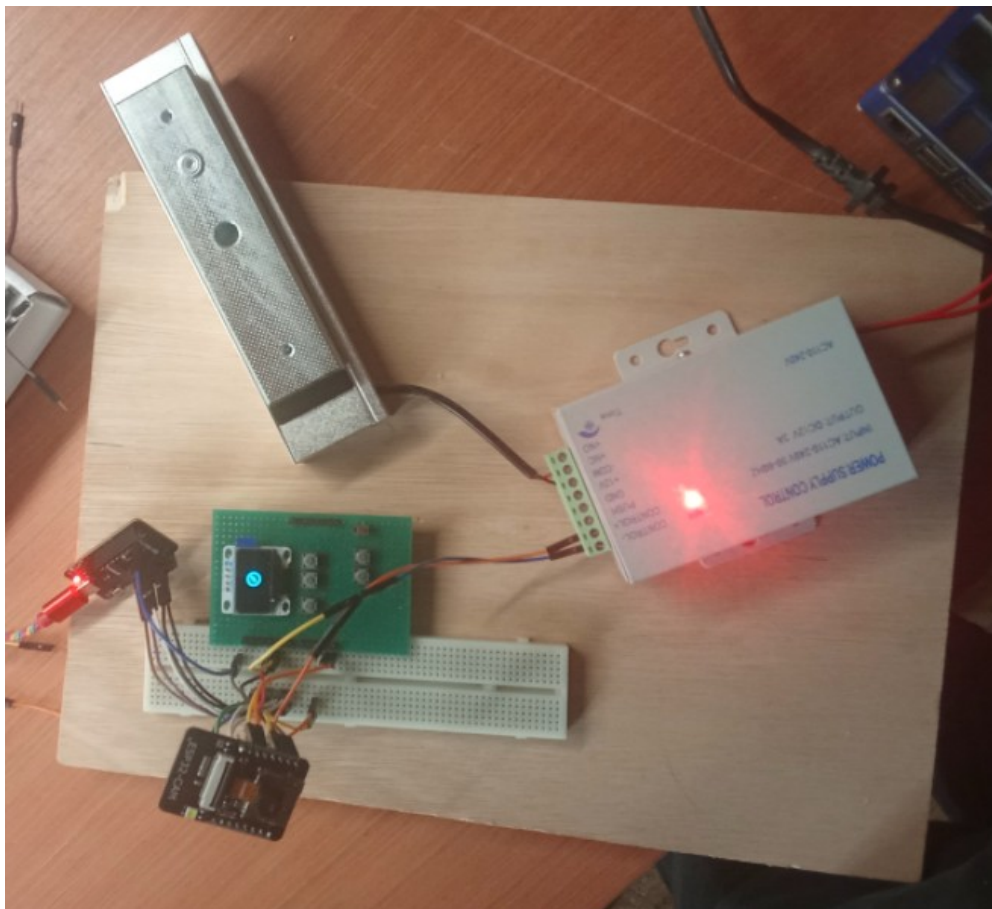
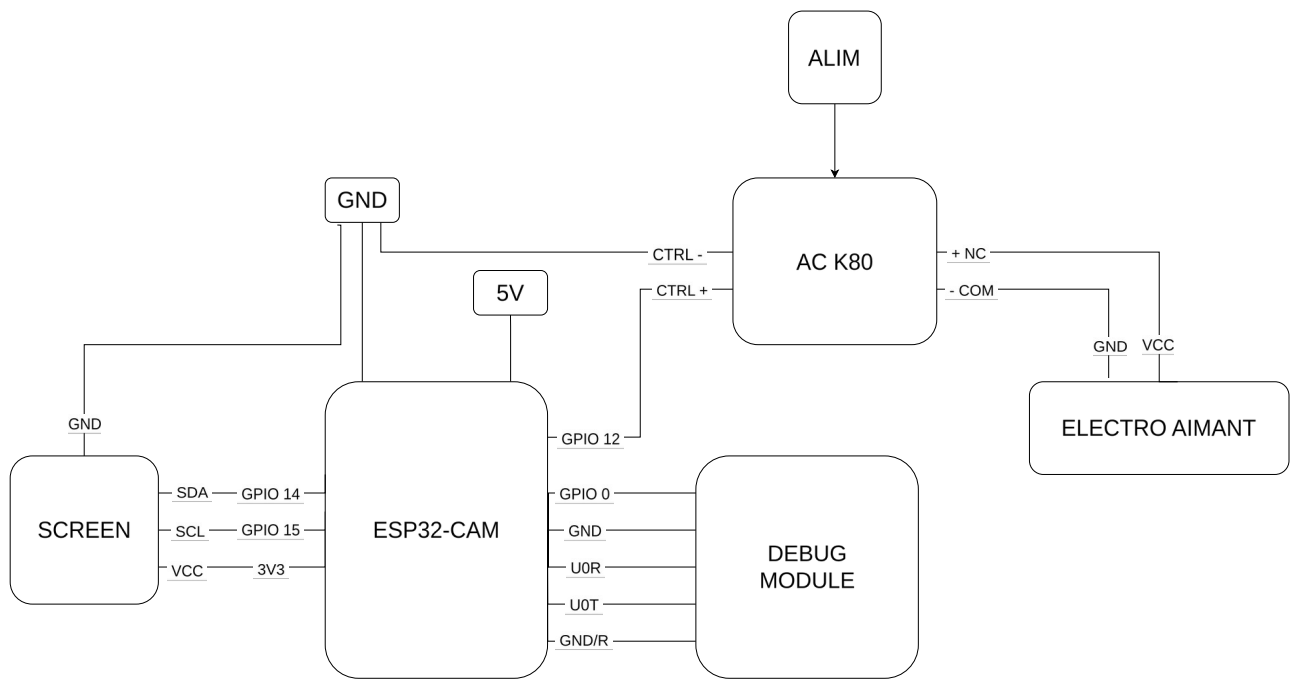
La serrure est commandée par une alimentation de contrôle LIBO Access Control Power Supply K80 (12 V / 3 A), spécialement conçue pour les systèmes de contrôle d'accès. La serrure électromagnétique est maintenue sous tension en fonctionnement normal afin de rester verrouillée. Lorsqu'une demande d'ouverture est validée par le serveur, l'alimentation de la serrure est momentanément interrompue, ce qui libère le mécanisme de verrouillage pendant une durée déterminée avant le retour à l'état sécurisé.

Afin de faciliter les interactions avec l'utilisateur, le prototype intègre un écran OLED connecté à l'ESP32. Celui-ci affiche les différents états du système, tels que le démarrage, la connexion au réseau Wi-Fi, la configuration, l'attente d'un QR code ou encore le résultat d'une demande d'ouverture. L'interface matérielle est volontairement limitée afin de réduire la complexité du dispositif.

La configuration initiale de la serrure est réalisée exclusivement par lecture de QR codes. Lors du premier démarrage, le firmware vérifie la présence des paramètres indispensables à son fonctionnement, notamment les informations de connexion au réseau Wi-Fi et la configuration du serveur backend. Si l'une de ces informations est absente, la serrure bascule automatiquement en mode configuration et attend la lecture des QR codes correspondants.

Le prototype comporte également un unique bouton permettant la réinitialisation complète de la configuration. Un appui maintenu pendant trois secondes déclenche la suppression des paramètres enregistrés et provoque le redémarrage du dispositif.

Une fois le fonctionnement du prototype validé, une seconde version a été réalisée sur un circuit imprimé soudé. Les différents composants ont été intégrés dans un boîtier fixé directement sur une porte afin de reproduire des conditions d'utilisation proches d'un environnement réel.



6.1.2 Développement du firmware embarqué

Le firmware embarqué constitue le cœur du système de contrôle d'accès. Il est responsable de l'ensemble des traitements réalisés par la serrure connectée, depuis la lecture d'un QR code jusqu'à la commande d'ouverture de la serrure électromagnétique, en passant par les communications avec le serveur central. Son développement a été réalisé en langage C en s'appuyant sur le système d'exploitation temps réel FreeRTOS.

Le choix de FreeRTOS répond à la nécessité de gérer plusieurs traitements indépendants de manière concurrente. Un système de contrôle d'accès doit être capable d'assurer simultanément l'acquisition des QR codes, l'affichage de l'état du dispositif, l'exécution de services annexes ainsi que la logique de contrôle du système, sans qu'une opération longue ou bloquante ne perturbe l'ensemble de l'application.

L'architecture logicielle repose sur quatre tâches principales exécutées en parallèle. Chacune possède une responsabilité clairement définie et ne réalise qu'un ensemble limité d'opérations. Cette séparation des responsabilités permet de limiter le couplage entre les différents modules et d'obtenir une architecture plus modulaire.

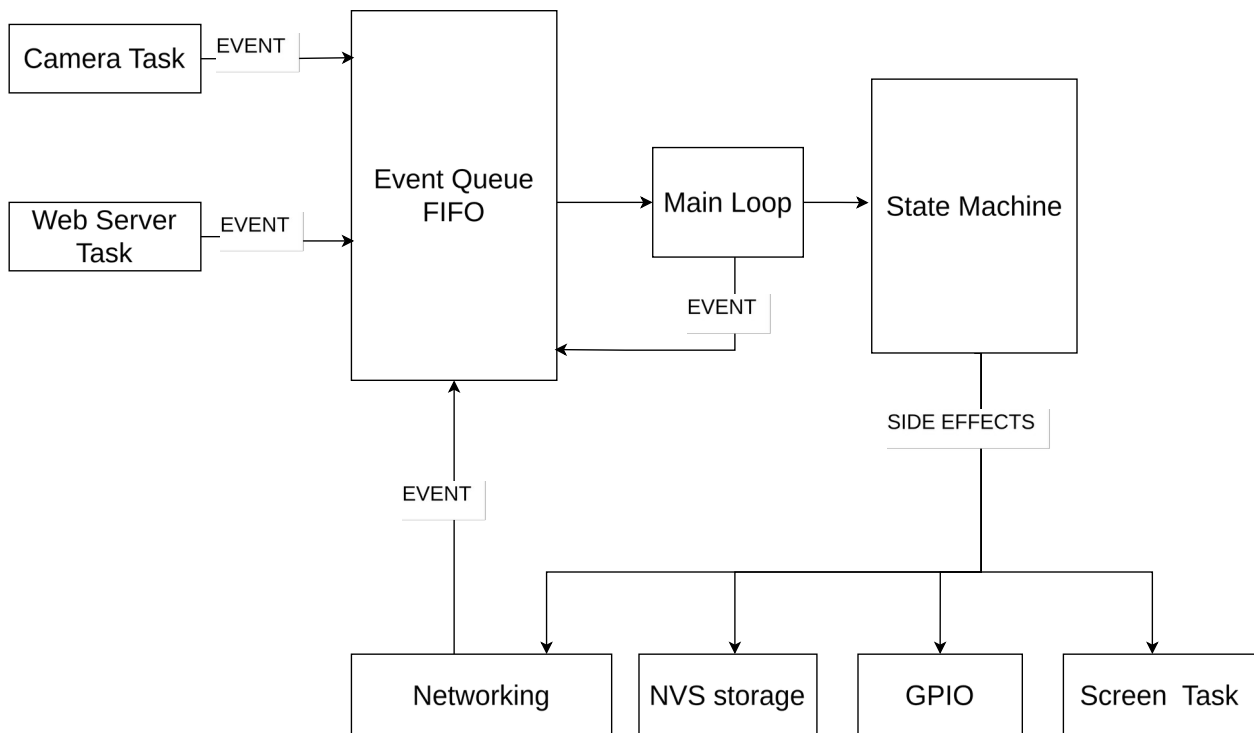
La première tâche est dédiée à la gestion de la caméra. Elle pilote le capteur de l'ESP32-CAM, réalise l'acquisition des images et décode les QR codes. Lorsqu'un QR code est reconnu, la tâche ne prend aucune décision concernant sa validité. Elle génère simplement un événement `EVENT_QR_READ` contenant les informations extraites avant de le transmettre à la file de messages.

Une seconde tâche est consacrée à la gestion de l'interface utilisateur. Elle pilote l'écran OLED et affiche en permanence l'état courant du dispositif. Les différentes phases du fonctionnement, telles que le démarrage, la configuration, la connexion au serveur, l'attente d'un QR code ou encore le résultat d'une demande d'accès, sont ainsi présentées à l'utilisateur sans impacter les autres traitements réalisés par le firmware.

Une troisième tâche exécute un serveur web embarqué. Celui-ci est utilisé pour certaines opérations spécifiques de maintenance ou d'administration du dispositif (comme l'ouverture à distance ou le reset de la serrure).

La dernière tâche correspond à la boucle principale (*Main Loop*), qui constitue le véritable orchestrateur du système. Contrairement aux autres tâches, elle n'est pas associée à un périphérique matériel particulier. Son rôle consiste à centraliser l'ensemble des traitements décisionnels du firmware et à coordonner les différents modules.

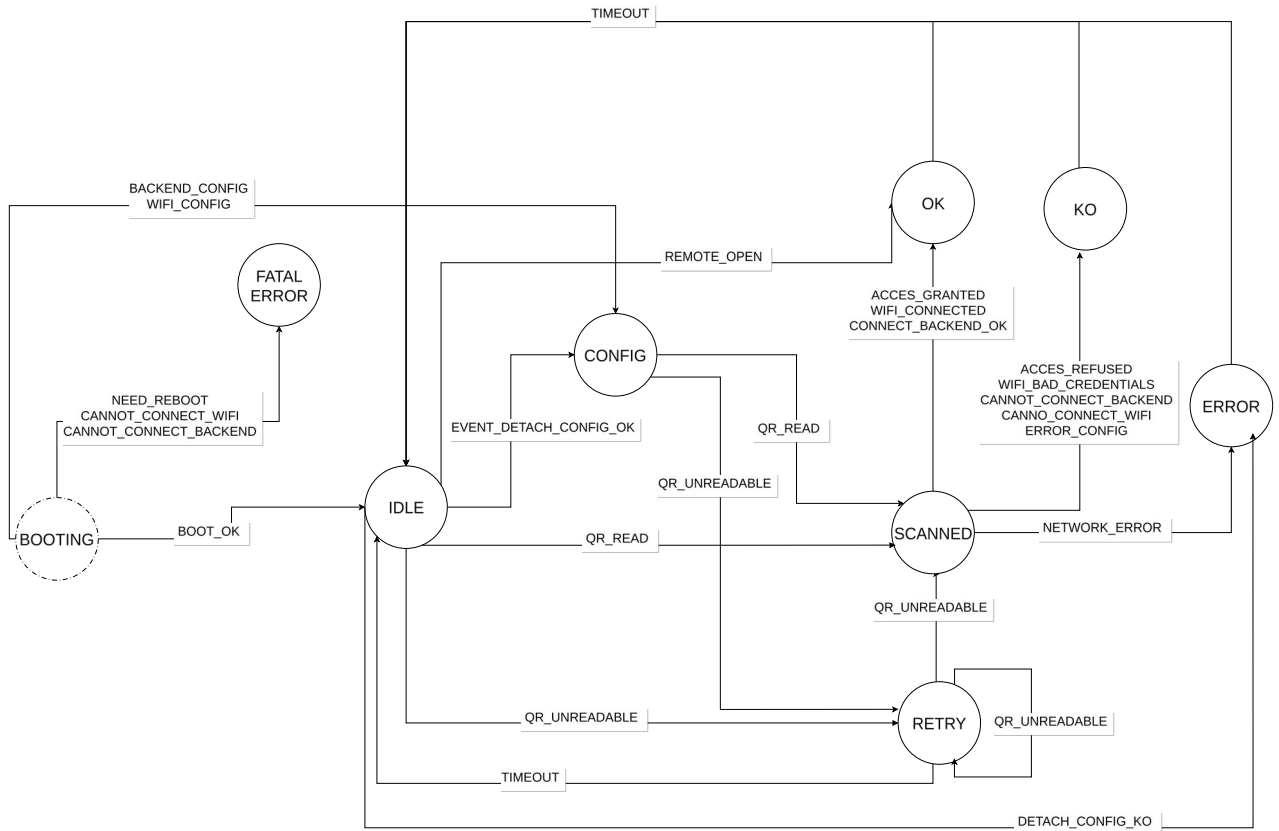
Afin de limiter les dépendances entre les tâches, celles-ci ne communiquent jamais directement entre elles. Chaque module produit uniquement des événements représentant les actions qu'il détecte, comme la lecture d'un QR code, la réception d'une commande ou la fin d'une opération. Ces événements sont insérés dans une file de messages (*Event Queue*) fournie par FreeRTOS. Fonctionnant selon le principe FIFO (*First In, First Out*), cette file constitue l'unique point de communication entre les différents composants du firmware.



La Main Loop consomme en permanence les événements présents dans cette file. Chaque événement est interprété puis transmis à une machine à états finis (*Finite State Machine*), responsable de déterminer le comportement du système. Cette organisation permet de centraliser l'ensemble de la logique métier dans un seul composant, tandis que les autres tâches restent entièrement dédiées à leur domaine fonctionnel.

La machine à états pilote l'ensemble du cycle de vie de la serrure connectée. Lors du démarrage, elle vérifie tout d'abord la présence des paramètres enregistrés dans la mémoire non volatile (NVS). Si aucune configuration Wi-Fi n'est disponible, le système bascule automatiquement dans un mode de configuration dédié. Une fois les paramètres réseau enregistrés, le même mécanisme est appliqué à la configuration du backend. Lorsque l'ensemble des informations nécessaires est disponible, la serrure établit sa connexion au réseau local puis au serveur central avant d'entrer dans son mode de fonctionnement normal.

En fonctionnement normal, la machine à états attend les événements générés par les différents modules. Lorsqu'un événement `EVENT_QR_READ` est reçu, la boucle principale réalise une requête HTTP vers le backend afin de vérifier la validité du QR code, la période de réservation ainsi que les autorisations d'accès associées. La réponse du serveur détermine ensuite la transition suivante de la machine à états. En cas d'autorisation, la serrure électromagnétique est déverrouillée pendant la durée prévue avant de revenir automatiquement à son état sécurisé. En cas de refus, le système informe simplement l'utilisateur et retourne en attente d'un nouveau QR code. Les erreurs de communication, les pertes de connexion réseau ainsi que les différents délais d'attente sont également pris en charge par cette machine à états, garantissant un comportement cohérent quelles que soient les conditions d'utilisation.



Cette architecture orientée événements présente plusieurs avantages. La séparation des responsabilités permet de conserver un code plus lisible et plus facilement maintenable. Le découplage entre les tâches réduit les interactions directes entre les différents modules et limite les risques liés aux accès concurrents aux ressources du microcontrôleur. Enfin, la centralisation de la logique décisionnelle dans la Main Loop garantit que toutes les décisions importantes — validation des accès, communications avec le backend, gestion de la configuration ou transitions de la machine à états — sont prises en un point unique. Cette organisation facilite non seulement le débogage et les évolutions futures, mais constitue également une architecture robuste et adaptée aux contraintes d'un système embarqué destiné au contrôle d'accès connecté.

6.1.3 Développement de l'interface graphique embarquée

L'interface utilisateur de la serrure est assurée par un écran OLED permettant d'afficher les différentes étapes du fonctionnement du prototype, telles que la configuration initiale, les messages d'état ou le résultat d'une demande d'accès.

Lors des premières phases de développement, l'affichage reposait directement sur la bibliothèque **U8g2**, utilisée comme pilote de l'écran OLED. Si cette bibliothèque permet de piloter efficacement l'affichage, elle se limite essentiellement à des primitives de dessin de bas niveau et ne fournit pas d'outils facilitant le développement d'interfaces graphiques plus élaborées.

Afin de disposer d'un environnement de développement graphique plus riche, un **portage partiel de la bibliothèque raylib** a été réalisé pour l'ESP32. L'objectif n'était pas uniquement de remplacer les fonctions de dessin de U8g2, mais surtout de bénéficier de l'écosystème proposé par raylib, en particulier de la bibliothèque **raymath**.

L'utilisation de raymath a permis d'employer des structures et des opérations mathématiques de plus haut niveau, telles que les vecteurs, les matrices ou les quaternions, simplifiant ainsi les calculs nécessaires au développement de l'interface graphique. Cette approche offre une abstraction plus moderne que les calculs réalisés directement sur des coordonnées entières et facilite l'implémentation d'animations, de transformations géométriques ou de futurs éléments graphiques plus évolués.

Bien que le prototype n'exploite qu'une partie des fonctionnalités offertes par raylib, cette adaptation constitue une base réutilisable pour de futurs développements embarqués nécessitant une interface graphique plus avancée.

6.2 Développement logiciel

6.2.1 Architecture générale

Le backend constitue le point central de l'ensemble du système de contrôle d'accès. Il assure l'interface entre les différents acteurs de la plateforme : le tableau de bord d'administration, les

serrures connectées ainsi que la base de données. Toutes les décisions liées à la gestion des accès sont prises à ce niveau, ce qui garantit un comportement cohérent de l'ensemble du système.

Le serveur a été développé avec **Node.js** en utilisant le framework **Express**. Ce choix permet de disposer d'un environnement léger, performant et particulièrement adapté au développement d'une API REST. Afin d'organiser le code de manière claire et évolutive, une architecture **MVC (Model - View - Controller)** a été adoptée. Les routes assurent le routage des requêtes, les contrôleurs orchestrent les traitements métier tandis que les modèles, implémentés avec **Mongoose**, assurent les interactions avec la base de données.

Les échanges entre les différents composants reposent sur une API REST utilisant le format **JSON**. Cette approche facilite la communication entre des clients de nature différente, qu'il s'agisse du tableau de bord web ou du firmware embarqué sur les ESP32.

L'architecture générale de l'application est illustrée par la figure suivante.

Figure X : Architecture générale du logiciel

6.2.2 Base de données

La persistance des données est assurée par **MongoDB**, une base de données orientée documents utilisée au travers de **Mongoose**. Ce choix s'est révélé particulièrement adapté au projet en raison de la structure des données manipulées et de leur proximité avec le format JSON utilisé par l'API.

Chaque ressource principale de l'application est représentée par une collection dédiée.

Collection	Description
Users	Comptes utilisateurs
Locks	Serrures connectées
Zones	Zones d'accès
Passes	QR codes d'accès
Events	Journal des événements

L'utilisation de Mongoose permet de définir les modèles de données ainsi que leurs règles de validation, tout en simplifiant les opérations de lecture et d'écriture dans la base. Cette organisation facilite également les évolutions futures du projet, l'ajout de nouvelles propriétés ne nécessitant que peu de modifications de la structure existante.

Figure X : Modèle de données MongoDB

6.2.3 Interface d'administration

Afin de faciliter l'administration du système, une interface web a été développée avec **Vue.js**. Celle-ci permet de centraliser l'ensemble des opérations de gestion sans nécessiter d'accès direct à la base de données.

Depuis cette interface, un administrateur peut créer et gérer les utilisateurs, enregistrer de nouvelles serrures, organiser celles-ci en zones, générer les QR codes de configuration destinés à leur installation ainsi que consulter l'historique des événements produits par le système.

L'ensemble des opérations réalisées depuis le tableau de bord transite exclusivement par l'API du backend. Cette organisation garantit que toutes les règles métier sont appliquées de manière uniforme, quel que soit le client utilisé.

Figure X : Tableau de bord d'administration

6.2.4 Gestion des accès et communication avec les serrures

Le backend est responsable de toute la logique liée au contrôle d'accès. Lorsqu'une nouvelle serrure est créée depuis le tableau de bord, le serveur génère automatiquement son identifiant ainsi qu'un **token d'authentification** unique. Ces informations sont ensuite regroupées dans un QR code de configuration destiné à être scanné lors de l'installation de la serrure.

Au premier démarrage, le firmware détecte l'absence de configuration et passe automatiquement en mode initialisation. Après lecture du QR code, les paramètres de connexion ainsi que le token sont enregistrés dans la mémoire non volatile du microcontrôleur. Cette opération ne doit être réalisée qu'une seule fois, les informations étant ensuite conservées entre les redémarrages.

Contrairement aux utilisateurs de l'application web, les serrures utilisent un **token fixe** plutôt qu'un mécanisme d'authentification basé sur des JWT. Une serrure possédant une identité permanente connue du backend, un simple token aléatoire est suffisant pour authentifier chacune de ses requêtes. Cette solution réduit la complexité du firmware tout en restant adaptée aux besoins du projet.

Lorsqu'un utilisateur présente un QR code devant la serrure, celui-ci est décodé puis transmis au backend via une requête HTTP authentifiée. Le serveur vérifie alors la validité du QR code, sa période d'utilisation ainsi que les autorisations associées avant de renvoyer une réponse au firmware. Si l'accès est autorisé, la serrure est déverrouillée pendant la durée prévue ; dans le cas contraire, la demande est simplement refusée.

Chaque opération est enregistrée dans la collection **Events**, permettant de conserver un historique complet des accès, des refus et des différentes actions réalisées sur le système. Cette centralisation de la logique métier garantit que toutes les décisions sont prises par le backend, tandis que les serrures restent limitées à l'exécution des actions qui leur sont demandées.

Figure X : Diagramme de séquence d'une demande d'accès

6.2.5 Communication temps réel

Le tableau de bord d'administration utilise des **WebSockets** afin d'afficher en temps réel les événements produits par le système. Chaque ouverture de serrure, création d'un utilisateur ou nouvel événement enregistré est immédiatement transmis au navigateur sans nécessiter de rafraîchissement manuel de la page.

L'utilisation de WebSockets a également été envisagée pour les communications avec les serrures connectées. Les essais réalisés ont toutefois montré que cette solution sollicitait fortement les ressources limitées de l'ESP32. Le maintien simultané d'une connexion persistante, de la caméra, des différentes tâches FreeRTOS et des autres traitements embarqués entraînait une consommation mémoire importante ainsi que des problèmes de stabilité.

Pour cette raison, les communications entre les serrures et le backend reposent exclusivement sur des requêtes HTTP initiées par les dispositifs eux-mêmes. Cette architecture répond pleinement aux besoins du projet tout en conservant un firmware plus simple, plus robuste et plus adapté aux contraintes matérielles de l'ESP32.

6.3 Tests et validation

6.3.1 Validation du firmware

Les premiers tests ont porté sur le bon fonctionnement du firmware embarqué. Chaque fonctionnalité a été validée indépendamment avant d'être intégrée au reste du système.

Les essais réalisés ont notamment permis de vérifier :

- le démarrage du microcontrôleur ;
- le passage automatique en mode configuration lors de l'absence de paramètres ;
- la lecture des QR codes de configuration ;
- la sauvegarde des paramètres dans la mémoire NVS ;
- la reconnexion automatique après un redémarrage ;
- la lecture des QR codes utilisateurs ;
- le pilotage de la serrure électromagnétique.

Ces essais ont permis de valider le comportement de la machine à états ainsi que les interactions entre les différentes tâches FreeRTOS.

6.3.2 Validation du backend

Le backend a été testé au moyen de requêtes HTTP ainsi qu'à travers l'utilisation du tableau de bord d'administration.

Les principaux tests réalisés sont les suivants :

- Création d'une serrure
- Generation des QR codes de configurations
- Validation / refus des QR codes utilisateurs
- Enregistrement des événements
- Consultation des logs des événements

Ces tests ont permis de vérifier le bon fonctionnement de l'API ainsi que la cohérence des données enregistrées dans la base MongoDB.

6.3.3 Validation du système complet

Une fois les différents composants validés individuellement, plusieurs essais ont été réalisés afin de vérifier le fonctionnement global du prototype.

Le scénario de test principal est le suivant :

1. Création d'une nouvelle serrure depuis le tableau de bord.
2. Génération du QR code de configuration.
3. Installation de la serrure et lecture du QR.
4. Connexion au réseau Wi-Fi.
5. Connexion au backend.
6. Présentation d'un QR code utilisateur.
7. Validation des droits d'accès.
8. Ouverture de la serrure.
9. Enregistrement de l'événement.
10. Mise à jour du tableau de bord en temps réel.

Ce scénario a permis de valider l'ensemble de la chaîne de traitement, depuis la configuration initiale jusqu'à l'ouverture physique de la serrure.

6.4.4 Gestion des cas d'erreur

Des essais ont également été réalisés afin de vérifier le comportement du système en présence de situations anormales.

Les cas suivants sont parfaitement gérés par le système :

- QR code utilisateur invalide
- Perte de connexion wifi
- Backend indisponible
- Redémarrage de l'ESP32
- Token de serrure invalide
- QR code de configuration invalide

Ces différents essais ont permis de vérifier que le système restait dans un état cohérent même en présence d'erreurs de communication ou de configuration.

7. Technologies et outils utilisés

Le développement du prototype a mobilisé différents outils logiciels et matériels couvrant le développement embarqué, le développement web ainsi que la gestion du projet. Le tableau ci-dessous présente les principales technologies utilisées au cours du stage.

Catégorie	Technologies / Outils
Langages	C, JavaScript, TypeScript
Développement embarqué	Arduino framework, FreeRTOS, PlatformIO
Backend	Node.js, Express
Frontend	Vue.js
Base de données	MongoDB
Communication	REST, HTTP, WebSocket
Gestion de versions	Git, Forgejo
Gestion de projet	Planka
Tests API	Insomnia
Environnement de développement	Visual Studio Code
Système d'exploitation	Linux
Matériel	ESP32-CAM, écran OLED, serrure électromagnétique, alimentation LIBO K80, breadboard, PCB à pastilles

Les principales bibliothèques utilisées au cours du développement sont présentées dans le tableau suivant.

Bibliothèque	Rôle
FreeRTOS	Gestion des tâches concurrentes du firmware
ESP32QRCodeReader	Acquisition et décodage des QR codes
ArduinoJson	Sérialisation et désérialisation des données JSON
raylib (portage)	Couche d'abstraction graphique développée pour le prototype
raymath	Calculs vectoriels et opérations mathématiques
U8g2	Pilotage de l'écran OLED
Fetch	Requêtes HTTP du frontend vers le backend
Socket.IO	Communication temps réel entre le backend et le tableau de bord
Mongoose	Modélisation des données MongoDB

L'utilisation de ces technologies a permis de développer une architecture complète couvrant l'ensemble des composants du système, depuis le firmware embarqué jusqu'à l'interface d'administration, tout en facilitant la maintenance et l'évolution du prototype.

8. Difficultés rencontrées et solutions apportées

Le développement du prototype a nécessité plusieurs ajustements techniques au cours du stage. Les difficultés rencontrées ont principalement concerné les contraintes matérielles de l'ESP32, l'organisation du firmware ainsi que l'amélioration progressive de l'expérience utilisateur. Chaque difficulté a conduit à une réflexion sur l'architecture du système et, dans plusieurs cas, à une évolution des choix de conception initiaux.

8.1 Communication temps réel avec les serrures

L'une des premières pistes explorées consistait à utiliser une communication **WebSocket** entre le backend et les serrures afin de maintenir une connexion persistante avec le serveur. Cette approche devait notamment permettre au backend d'envoyer directement des commandes vers les dispositifs sans attendre une requête de leur part.

Plusieurs implémentations ont été expérimentées, notamment **Socket.IO** puis la bibliothèque **esp_websocket_client** de l'ESP-IDF. Bien que ces solutions fonctionnent sur des cas simples, leur utilisation conjointe avec les autres composants du firmware, en particulier la caméra, les différentes tâches FreeRTOS et la pile réseau, augmentait significativement la consommation des ressources de l'ESP32. La gestion d'une connexion WebSocket permanente introduisait également une complexité supplémentaire dans la gestion des états de connexion.

Après plusieurs essais, cette architecture a été abandonnée au profit d'un modèle plus simple dans lequel les serrures initient elles-mêmes les communications via des requêtes HTTP. Les WebSockets ont été conservés uniquement pour la communication entre le backend et l'interface d'administration, où ils permettent une mise à jour instantanée du tableau de bord sans impacter les contraintes matérielles des dispositifs embarqués.

Ce choix a permis d'obtenir un firmware plus stable tout en conservant les fonctionnalités temps réel nécessaires à l'administration du système.

8.2 Organisation des tâches FreeRTOS

Le firmware repose sur plusieurs tâches exécutées en parallèle, notamment pour la gestion de la caméra, de l'interface graphique, du serveur web embarqué ainsi que de la boucle principale du système.

Cette architecture améliore la réactivité du prototype mais complique le raisonnement sur le déroulement du programme. Les différentes tâches doivent partager certaines ressources et communiquer entre elles sans provoquer de conflits ou de comportements imprévisibles.

Afin de limiter cette complexité, une architecture orientée événements a été mise en place. Les différentes tâches ne réalisent que les opérations spécifiques à leur domaine de responsabilité et transmettent les événements qu'elles produisent à une file de messages (**queue**) FreeRTOS. Une machine à états centrale, exécutée dans la boucle principale, est ensuite chargée d'interpréter ces événements et de décider des actions à entreprendre.

Cette organisation présente plusieurs avantages. Les échanges entre les modules sont centralisés, les traitements critiques restent séquentiels et le comportement global du système est plus simple à comprendre et à maintenir malgré l'utilisation d'un environnement multitâche.

8.3 Une configuration pensée pour l'utilisateur

Lors des premières réflexions sur le projet, la configuration des serrures reposait sur une approche classique consistant à renseigner les paramètres directement dans le firmware avant sa compilation. Chaque modification du réseau Wi-Fi ou de la configuration du backend nécessitait alors une recompilation puis un nouveau flash du microcontrôleur.

Cette méthode s'est rapidement révélée peu adaptée à un système destiné à être installé sur plusieurs dispositifs. La configuration devenait fastidieuse et nécessitait systématiquement l'intervention d'une personne disposant des outils de développement.

Une autre approche a donc été imaginée : utiliser des **QR codes comme interface de configuration**. Lorsqu'une serrure est démarrée sans paramètres enregistrés, elle passe automatiquement en mode configuration et attend la lecture des QR codes contenant les informations nécessaires à son initialisation.

Cette solution transforme la caméra en véritable interface utilisateur. La configuration peut être réalisée directement sur le lieu d'installation sans connexion physique à l'ESP32 ni recompilation du firmware. Elle simplifie considérablement le déploiement du système tout en offrant une expérience d'installation plus intuitive.

8.4 Évolution du prototype matériel

Les premières versions du prototype ont été réalisées sur une **breadboard**, afin de faciliter les essais et les nombreuses modifications du câblage durant les phases de développement. Cette approche permettait de remplacer rapidement un composant ou de modifier les connexions au fur et à mesure de l'évolution du projet.

Une fois l'architecture matérielle validée, un montage plus robuste a été réalisé sur une **plaque à pastilles**. Les composants ont été soudés de manière permanente puis intégrés dans un boîtier en carton conçu spécifiquement pour être fixé sur une porte.

Bien que ce boîtier ne constitue pas une solution industrielle, il a permis de disposer d'un prototype fonctionnel, facilement transportable et suffisamment robuste pour réaliser les démonstrations du système dans des conditions proches d'une installation réelle.

8.5 Développement d'une couche graphique embarquée

L'interface graphique du prototype reposait initialement sur la bibliothèque **U8g2**, utilisée pour piloter l'écran OLED. Bien que cette bibliothèque remplisse parfaitement son rôle de pilote d'affichage, elle offre principalement des primitives graphiques de bas niveau.

Parallèlement au stage, plusieurs projets personnels de développement de jeux vidéo réalisés avec **raylib** ont permis de découvrir une approche plus moderne de la programmation graphique. Cette bibliothèque fournit non seulement une API de dessin homogène, mais également un ensemble d'outils mathématiques, notamment grâce au module **raymath**, facilitant la manipulation de vecteurs, de matrices et d'autres structures géométriques.

Dans le but de proposer une interface plus évoluée et de sortir des interfaces minimalistes souvent rencontrées sur les objets connectés, un **portage partiel de raylib** a été réalisé pour l'écran OLED utilisé par le prototype. Ce portage adapte les principales fonctions graphiques de la bibliothèque au pilote matériel existant et permet de bénéficier d'une couche d'abstraction plus riche tout en conservant les performances nécessaires au fonctionnement de l'ESP32.

Au-delà de l'aspect technique, cette initiative a permis de rendre le développement de l'interface plus agréable et plus modulaire. Elle constitue également une base réutilisable pour de futurs projets embarqués utilisant des écrans graphiques.

Conclusion

Les difficultés rencontrées durant le stage n'ont pas uniquement conduit à résoudre des problèmes techniques ; elles ont également permis de faire évoluer l'architecture du projet et de remettre en question certains choix initiaux. Les contraintes propres au développement embarqué ont nécessité de trouver un équilibre entre performances, simplicité de mise en œuvre et facilité de maintenance. Cette démarche d'amélioration continue a contribué à aboutir à un prototype plus robuste, plus cohérent et mieux adapté à une utilisation en conditions réelles.

9. Bilan et conclusion

9.1 Bilan du projet

Au cours de ce stage, un prototype complet de serrure connectée a été conçu et développé afin de répondre aux besoins de l'entreprise en matière de contrôle d'accès. L'objectif principal était de

mettre en place une architecture permettant l'administration centralisée de serrures connectées, tout en proposant une solution simple à déployer et évolutive.

Le prototype réalisé intègre l'ensemble des composants nécessaires au fonctionnement du système. Le firmware embarqué sur ESP32 assure la gestion du matériel, la lecture des QR codes et la communication avec le backend. Celui-ci centralise la logique métier, gère l'authentification des utilisateurs et des serrures, valide les demandes d'accès et conserve l'historique des événements. Enfin, une interface web permet aux administrateurs de gérer les utilisateurs, les serrures et les différentes opérations du système.

Les objectifs définis au début du stage ont été atteints. Il est désormais possible de configurer une serrure grâce à des QR codes, d'autoriser ou de refuser un accès en fonction des informations enregistrées dans la base de données, de piloter l'ensemble du système depuis une interface d'administration et de consulter les événements générés par les différents dispositifs.

Au-delà des fonctionnalités développées, ce projet a également permis d'expérimenter plusieurs approches techniques avant d'aboutir à l'architecture retenue. Les contraintes propres au développement embarqué ont notamment conduit à faire évoluer certains choix initiaux, comme l'abandon des WebSockets sur les ESP32 au profit d'une communication HTTP initiée par les serrures. De la même manière, la mise en place d'une architecture événementielle reposant sur une machine à états et une file de messages a permis de simplifier l'organisation du firmware malgré l'utilisation de plusieurs tâches FreeRTOS.

Le prototype constitue aujourd'hui une base solide pouvant servir à de futurs développements. Plusieurs évolutions pourraient être envisagées, telles que l'intégration d'un mécanisme de mise à jour à distance du firmware (OTA), la conception d'un circuit imprimé dédié ou encore l'ajout de nouveaux moyens d'authentification et d'une gestion plus fine des permissions d'accès.

9.2 Bilan personnel

Ce stage m'a offert l'opportunité de participer au développement d'un projet complet, couvrant aussi bien les aspects logiciels que matériels. Contrairement aux projets réalisés dans le cadre de la formation, j'ai été amené à concevoir une solution destinée à répondre à un besoin concret, en tenant compte des contraintes techniques, des choix d'architecture et des méthodes de travail utilisées en entreprise.

Sur le plan technique, cette expérience m'a permis de renforcer mes compétences dans des domaines variés. J'ai approfondi mes connaissances en développement embarqué avec l'ESP32, ESP-IDF et FreeRTOS, tout en consolidant mes acquis en développement backend avec Node.js, Express et MongoDB. Le développement simultané du firmware, du backend et de l'interface d'administration m'a également permis d'appréhender la conception d'une architecture complète dans laquelle plusieurs technologies interagissent pour former un système cohérent.

Ce stage m'a également sensibilisé à l'importance des choix de conception. Plusieurs décisions prises au début du projet ont été remises en question afin de mieux répondre aux contraintes rencontrées. J'ai ainsi compris qu'une solution techniquement intéressante n'est pas nécessairement la plus adaptée, et qu'il est souvent préférable de privilégier une architecture simple, robuste et facilement maintenable.

Au-delà des compétences techniques, cette expérience m'a permis de découvrir l'organisation d'un projet informatique en entreprise. L'utilisation quotidienne de Git, Forgejo et Planka, les échanges réguliers avec mon maître de stage ainsi que le fonctionnement inspiré des méthodes Agile m'ont permis de développer une meilleure autonomie et d'adopter une approche plus rigoureuse dans la conduite d'un projet.

Enfin, ce stage a confirmé mon intérêt pour le développement de systèmes combinant logiciel, électronique et systèmes embarqués. J'ai particulièrement apprécié la diversité des problématiques abordées, allant de la programmation bas niveau sur microcontrôleur jusqu'au développement d'une application web complète. La possibilité de concevoir un système de bout en bout, puis de le voir fonctionner concrètement sur un prototype, a constitué une expérience particulièrement enrichissante et motivante pour la suite de mon parcours.